

Seal the Deal är en webbutik där användare kan "köpa" sälar. Applikationen är byggd som en single-page application med React och Vite i frontenden, och en REST-API med Express och MongoDB i backenden. Tanken var att bygga ett enkelt e-handelsflöde från grunden och få frontend och backend att fungera tillsammans som en helhet.

Frontenden är uppdelad i sidor, komponenter och context. Context används för att hålla i de delar som i princip hela applikationen behöver komma åt: varukorg, favoriter och inloggad användare. Utifrån hur en webbshops flöde behöver fungera, att till exempel kunna se samma information om till exempel varukorgen oavsett var i webbshoppen man befinner sig, så var det tydligt att den informationen behövde hanteras globalt. Hur var inte lika uppenbart från början. För applikationens kommunikation med backenden använder jag mig både av vanliga asynkrona funktioner och hooks. För inloggning och registrering där backenden endast behöver kallas vid knapptryck räckte det med en `authApi.js` med vanliga `async`-funktioner. Däremot för att kunna hämta information om sälarna och tidigare ordrar direkt när en komponent renderas passade hooks bättre. Just `useOrders` dock så är funktionerna i filen lite blandade. Den ena funktionen behöver vara en hook för när man loggat in och går in på profilsidan och ska kunna se tidigare ordrar. De andra två dock, att skapa en order och hämta specifik order, är egentligen bara vanliga `async`-funktioner som egentligen anropas vid ett knapptryck vid köp men eftersom alla tre funktioner hör samman så ville jag behålla dem i samma fil. Vad jag förstod efter att ha försökt kolla upp vad som är standard så skulle man både kunna behålla det så här eller dela upp det i två filer för att separera hooks från de vanliga funktionerna och att det ofta beror på storleken av applikationen.

Det som var mest utmanande med applikationen var ändå inte de enskilda delarna utan att hålla reda på helheten. Det är många delar att hålla koll på, mycket information från genomgångar som ska tänkas på samtidigt som man vill fokusera på en sak i taget. Att ha en relativt klar och tydlig design i Figma innan jag började koda var till stor hjälp. Varje designad del visade i princip vad som skulle synas och ungefär hur det skulle fungera, vilket gjorde att jag till viss del kunde "släppa" saker och förlita mig på designen. Men flödet och vilken data som behövdes hållas reda på behövde jag fortfarande tänka på hela tiden.

När jag började att bygga själva projektet så startade jag med frontend och använde mig av mockdata för att kunna testa användarflödet. Att använda mockdatan hjälpte mycket för att kunna fokusera på en komponent eller sida i taget och samtidigt kunna se resultatet med rätt data. Eftersom det dock skulle byggas på en backend sen så ville jag hela tiden försöka hålla i åtanke hur flödet skulle fungera då för att inte behöva arbeta om så många delar. Mycket fungerade dock likt men en sådan liten "enkel" sak som id-referenserna, som var många i projektet, behövdes bytas ut från `".id"` till `"._id"` för att det skulle fungera med MongoDB. En sådan liten detalj som påverkade jättemycket.

Sist var deployen en utmaning i sig, även om det inte var något som ingick i uppgiften. Frontend och backend är lagda separata i Render, frontend som en static site och backend som en Web Service. Att få frontend, backend och databas att fungera tillsammans live innebar bland annat att jag fick konfigurera miljövariabler i Render, uppdatera CORS-inställningarna så att de tillät den nya frontend-URL:en och öppna upp åtkomsten i MongoDB Atlas. Det är saker som är enklare att hantera lokalt och som behövde lite mer eftertanke när allt skulle köras på olika "platser" och prata med varandra över nätet.

En sak som blev lite av en överraskning där var hur React Router fungerar när sidan väl är ute. Eftersom appen är en single-page application finns det egentligen bara en HTML-fil som utgår från och Routern sköter sen vad som visas där. Det gjorde att allt fungerade så länge man klickade sig fram via menyn, men om man skrev in en undersida som /salar manuellt eller laddade om sidan (om det inte var home-page) fick man bara upp "not found". Det visade sig att servern letade efter en fil som inte finns, den förväntade sig en separat fil utifrån URL:en men som inte finns eftersom allt går via Index.html. Jag försökte först med ett script i projektet som skulle säga åt servern att skicka tillbaka till Index.html oavsett URL, men detta fungerade inte riktigt. Därför testade jag sen att lägga till en regel i Render som gjorde samma sak, skickade tillbaka till index, och då fungerade det. Jag förstod efter att detta verkar vara ett vanligt "problem" när man arbetar med SPA men som vi inte har stött på när vi kört SPA-programm lokalt eftersom det har hanterats automatiskt av Vites dev-server, men som vid deploy behövde konfigureras.

Det jag tar med mig från projektet är att struktur, och framför allt planering, är minst lika viktig som själva koden. Det är inte de enskilda funktionerna som har varit svåra, utan att få allt att hänga ihop. Även om design-biten och Figma inte är mina favoritdelar så känner jag uppskattning för både verktyget och att skapa en tydlig design att kunna arbeta utifrån då det både hjälper genom att man slipper koda om så mycket men också då det nästan fungerar lite som dokumentation att kunna utgå från under projektets gång.

LÄNKAR

Repot: <https://github.com/ellenlinnea/seal-webbshop>

Figma: <https://www.figma.com/design/gNpSQd5U32T4jT6tzy52xN/Seal-the-deal-webbshopp?node-id=0-1&t=pgq4ygotaiadMq48-1>

Live-sidan: <https://seal-webbshop.onrender.com/>